EKART

REST API Documentation

Version 1.0

A Django REST Framework e-commerce API

1. Overview

Ekart is a full-featured e-commerce REST API built with Django REST Framework (DRF). It provides endpoints for product management, shopping carts, order processing, customer profiles, and user authentication. The API uses JWT (JSON Web Token) authentication and follows RESTful conventions throughout.

Base URL

```
http://localhost:8000
```

Key Technologies

- Python / Django 5.x with Django REST Framework
- JWT Authentication via Simple JWT + Djoser
- PostgreSQL / SQLite database
- Redis for caching and Celery task queue
- django-filter for advanced query filtering
- Pagination: 10 items per page (default)

2. Authentication

Ekart uses JWT Bearer tokens. Obtain a token by registering and logging in, then include it in the Authorization header of every protected request.

Register a New User

```
POST /auth/users/
```

Field	Type	Required	Description
`username`	`string`	**Yes**	Unique username
`email`	`string`	**Yes**	Unique email address
`password`	`string`	**Yes**	Password (validated by Django rules)
`first_name`	`string`	No	User's first name
`last_name`	`string`	No	User's last name

Obtain JWT Token

```
POST /auth/jwt/create/
```

Request body:

```
{ "username": "john", "password": "secret123" }
```

Response (200 OK):

```
{ "access": "<JWT_ACCESS_TOKEN>", "refresh": "<JWT_REFRESH_TOKEN>" }
```

Refresh an Access Token

```
POST /auth/jwt/refresh/
```

Request body:

```
{ "refresh": "<JWT_REFRESH_TOKEN>" }
```

Using the Token

Include the access token in every authenticated request:

```
Authorization: JWT <access_token>
```

ℹ *Access tokens expire after 1 day. Use the refresh endpoint to obtain a new access token without re-authenticating.*

3. Permission Levels

Field	Type	Required	Description
`Public`	–	–	No authentication required (GET requests on products/collections)
`Authenticated`	`JWT`	–	Valid JWT token required (cart, orders, customer profile)
`Admin`	`JWT+Staff`	–	is_staff=True required (create/update/delete products, manage customers)

4. Endpoint Reference

4.1 Products

Base path: /store/products/

Method	Endpoint	Description	Auth
GET	`/store/products/`	List all products (paginated)	Public
POST	`/store/products/`	Create a new product	Admin
GET	`/store/products/{id}/`	Retrieve a single product by ID	Public
PUT	`/store/products/{id}/`	Fully update a product	Admin
PATCH	`/store/products/{id}/`	Partially update a product	Admin
DELETE	`/store/products/{id}/`	Delete a product (blocked if in an order)	Admin

Product Object

Field	Type	Required	Description
`id`	`integer`	–	Auto-generated primary key
`title`	`string`	**Yes**	Product name (max 255 chars)
`description`	`string`	No	Long-form description (nullable)
`slug`	`string`	No	URL-friendly slug (defaults to '-')
`unit_price`	`decimal`	**Yes**	Price per unit (min 1, max 9999.99)
`price_with_tax`	`decimal`	–	Read-only. unit_price × 1.1
`inventory`	`integer`	**Yes**	Stock quantity
`collection`	`integer`	**Yes**	FK to Collection (by ID)
`image`	`object`	–	Read-only. First product image (id, image URL)

Query Parameters (filtering & search)

Field	Type	Required	Description
`collection_id`	`integer`	No	Filter by collection ID

Field	Type	Required	Description
`unit_price__lt`	`decimal`	No	Products with price less than value
`unit_price__gt`	`decimal`	No	Products with price greater than value
`search`	`string`	No	Full-text search on product title
`ordering`	`string`	No	Sort by unit_price or updated_at (prefix - for descending)
`page`	`integer`	No	Page number for pagination (10 per page)

Example:

```
GET /store/products/?collection_id=3&unit_price__lt=50&ordering=-unit_price
```

4.2 Product Images

Base path: /store/products/{product_id}/images/

Method	Endpoint	Description	Auth
GET	`/store/products/{product_id}/images/`	List images for a product	Public
POST	`/store/products/{product_id}/images/`	Upload an image (multipart)	Admin
GET	`/store/products/{product_id}/images/{id}/`	Retrieve a single image	Public
DELETE	`/store/products/{product_id}/images/{id}/`	Delete an image	Admin

ℹ *Images are uploaded as multipart/form-data. Maximum file size is enforced by the validate_file_size validator. Only image file types are accepted.*

4.3 Collections

Base path: /store/collections/

Method	Endpoint	Description	Auth
GET	`/store/collections/`	List all collections with product count	Public
POST	`/store/collections/`	Create a collection	Admin
GET	`/store/collections/{id}/`	Retrieve a single collection	Public
PUT	`/store/collections/{id}/`	Fully update a collection	Admin
PATCH	`/store/collections/{id}/`	Partially update a collection	Admin
DELETE	`/store/collections/{id}/`	Delete collection (blocked if it has products)	Admin

Collection Object

Field	Type	Required	Description
`id`	`integer`	–	Auto-generated primary key
`title`	`string`	**Yes**	Collection name (max 255 chars)
`products_count`	`integer`	–	Read-only. Number of products in this collection

4.4 Product Reviews

Base path: /store/products/{product_id}/reviews/

Method	Endpoint	Description	Auth
GET	`/store/products/{product_id}/reviews/`	List all reviews for a product	Public
POST	`/store/products/{product_id}/reviews/`	Submit a review	Public
GET	`/store/products/{product_id}/reviews/{id}/`	Retrieve a review	Public
PUT	`/store/products/{product_id}/reviews/{id}/`	Update a review	Public
DELETE	`/store/products/{product_id}/reviews/{id}/`	Delete a review	Public

Review Object

Field	Type	Required	Description
`id`	`integer`	–	Auto-generated primary key
`name`	`string`	**Yes**	Reviewer display name (max 255 chars)
`description`	`string`	**Yes**	Review body text
`date`	`datetime`	–	Read-only. Auto-set on creation

4.5 Carts

Base path: /store/carts/

Method	Endpoint	Description	Auth
POST	`/store/carts/`	Create a new anonymous cart	Public
GET	`/store/carts/{uuid}/`	Retrieve a cart with all items	Public
DELETE	`/store/carts/{uuid}/`	Delete a cart	Public

ℹ *Cart IDs are UUID v4 strings (e.g. 550e8400-e29b-41d4-a716-446655440000). Store this UUID client-side — it acts as the anonymous session token.*

Cart Object

Field	Type	Required	Description
`id`	`uuid`	–	UUID primary key (auto-generated)
`items`	`array`	–	Array of CartItem objects

Field	Type	Required	Description
`total_price`	`decimal`	–	Read-only. Sum of all item totals

4.6 Cart Items

Base path: /store/carts/{cart_uuid}/items/

Method	Endpoint	Description	Auth
GET	`/store/carts/{uuid}/items/`	List items in a cart	Public
POST	`/store/carts/{uuid}/items/`	Add a product to the cart	Public
GET	`/store/carts/{uuid}/items/{id}/`	Retrieve a specific cart item	Public
PATCH	`/store/carts/{uuid}/items/{id}/`	Update item quantity	Public
DELETE	`/store/carts/{uuid}/items/{id}/`	Remove item from cart	Public

Add Item Request Body (POST)

Field	Type	Required	Description
`product_id`	`integer`	Yes	ID of the product to add
`quantity`	`integer`	Yes	Quantity to add (min 1). If product is already in cart, quantity is added to existing.

Update Item Request Body (PATCH)

Field	Type	Required	Description
`quantity`	`integer`	Yes	New quantity for the item (min 1)

4.7 Orders

Base path: /store/orders/

Method	Endpoint	Description	Auth
GET	`/store/orders/`	List orders (own orders for users; all for admin)	Auth
POST	`/store/orders/`	Create an order from an existing cart	Auth
GET	`/store/orders/{id}/`	Retrieve a specific order	Auth

Create Order Request Body (POST)

Field	Type	Required	Description
`cart_id`	`uuid`	Yes	UUID of a non-empty cart. Cart is deleted after the order is created.

ℹ *Creating an order atomically converts the cart into an order record. The cart must exist and must not be empty.*

Order Object (Response)

Field	Type	Required	Description
`id`	`integer`	–	Auto-generated order ID
`customer`	`integer`	–	Customer ID associated with the order
`placed_at`	`datetime`	–	Timestamp when order was placed
`payment_status`	`string`	–	P (Pending) | C (Complete) | F (Failed)
`items`	`array`	–	Array of OrderItem objects (id, product, quantity, unit_price)

4.8 Customers

Base path: /store/customers/

Method	Endpoint	Description	Auth
GET	`/store/customers/`	List all customers	Admin
GET	`/store/customers/{id}/`	Retrieve a customer	Admin
PUT	`/store/customers/{id}/`	Update a customer	Admin
DELETE	`/store/customers/{id}/`	Delete a customer	Admin
GET	`/store/customers/me/`	Get current user's profile	Auth
PUT	`/store/customers/me/`	Update current user's profile	Auth

Customer Object

Field	Type	Required	Description
`id`	`integer`	–	Auto-generated primary key
`user_id`	`integer`	–	Linked auth user ID
`birth_date`	`date`	No	Customer date of birth (YYYY-MM-DD)
`membership`	`string`	No	B (Bronze, default) | S (Silver) | G (Gold)

4.9 Authentication Endpoints (Djoser)

Method	Endpoint	Description	Auth
POST	`/auth/users/`	Register new user	Public
GET	`/auth/users/me/`	Get current authenticated user	Auth
PUT	`/auth/users/me/`	Update current user profile	Auth
POST	`/auth/jwt/create/`	Obtain JWT access + refresh tokens	Public
POST	`/auth/jwt/refresh/`	Refresh access token	Public
POST	`/auth/jwt/verify/`	Verify token validity	Public

5. Example Workflows

5.1 Browse & Filter Products

No authentication required.

```
GET /store/products/?collection_id=2&unit_price__lt=100&ordering=unit_price
GET /store/products/?search=headphones
```

5.2 Shopping Flow (Guest to Order)

Step 1 — Create a cart:

```
POST /store/carts/
Response: { "id": "550e8400-e29b-41d4-a716-446655440000", "items": [],
"total_price": 0 }
```

Step 2 — Add items:

```
POST /store/carts/550e8400-e29b-41d4-a716-446655440000/items/
Body: { "product_id": 7, "quantity": 2 }
```

Step 3 — Register and log in:

```
POST /auth/users/
Body: { "username": "jane", "email": "jane@example.com", "password": "Str0ng!" }
POST /auth/jwt/create/
Body: { "username": "jane", "password": "Str0ng!" }
```

Step 4 — Place order (authenticated):

```
POST /store/orders/
Header: Authorization: JWT <access_token>
Body: { "cart_id": "550e8400-e29b-41d4-a716-446655440000" }
```

5.3 View & Manage Orders

Authenticated users see only their own orders. Admin staff see all orders.

```
GET /store/orders/
Header: Authorization: JWT <access_token>
```

6. Error Responses

Field	Type	Required	Description
`400 Bad Request`	–	–	Validation error — response body contains field-level error details
`401 Unauthorized`	–	–	Missing or invalid JWT token
`403 Forbidden`	–	–	Authenticated but insufficient permissions (e.g., non-admin writing products)
`404 Not Found`	–	–	Resource does not exist

Field	Type	Required	Description
`405 Not Allowed`	–	–	Operation blocked (e.g., deleting a product that is in an order)

Validation errors return JSON with field names as keys:

```
{ "unit_price": ["Ensure this value is greater than or equal to 1."] }
```

7. Pagination

All list endpoints return paginated results with 10 items per page.

```
{   "count": 53,   "next": "http://localhost:8000/store/products/?page=2",
"previous": null,   "results": [ ... ] }
```

ℹ *Use the next and previous URLs directly, or append ?page=N to any list endpoint.*

8. Local Setup

- Clone the repository and create a virtual environment
- Install dependencies: pip install -r requirements.txt
- Configure your database in storefront/settings/development.py
- Start Redis (required for caching and Celery): redis-server
- Apply migrations: python manage.py migrate
- Create a superuser: python manage.py createsuperuser
- Run the development server: python manage.py runserver
- (Optional) Start Celery worker: celery -A storefront worker -l info
- (Optional) Start Celery beat: celery -A storefront beat -l info

ℹ *CORS is pre-configured to allow requests from http://localhost:8001. Update CORS_ALLOWED_ORIGINS in settings for your frontend URL.*

9. Data Model Summary

The diagram below summarises the key relationships between models.

Model	Key Relationships
`User (core)`	One-to-one with Customer
`Customer`	FK → User; has many Orders
`Collection`	Has many Products
`Product`	FK → Collection; has many ProductImages, Reviews, OrderItems, CartItems
`ProductImage`	FK → Product

`Review`	FK → Product
`Cart`	UUID PK; has many CartItems
`CartItem`	FK → Cart, Product; unique together
`Order`	FK → Customer; has many OrderItems
`OrderItem`	FK → Order, Product